Software Engineering
Sem 2

Aditya Kumar

Contents

Chapter 1

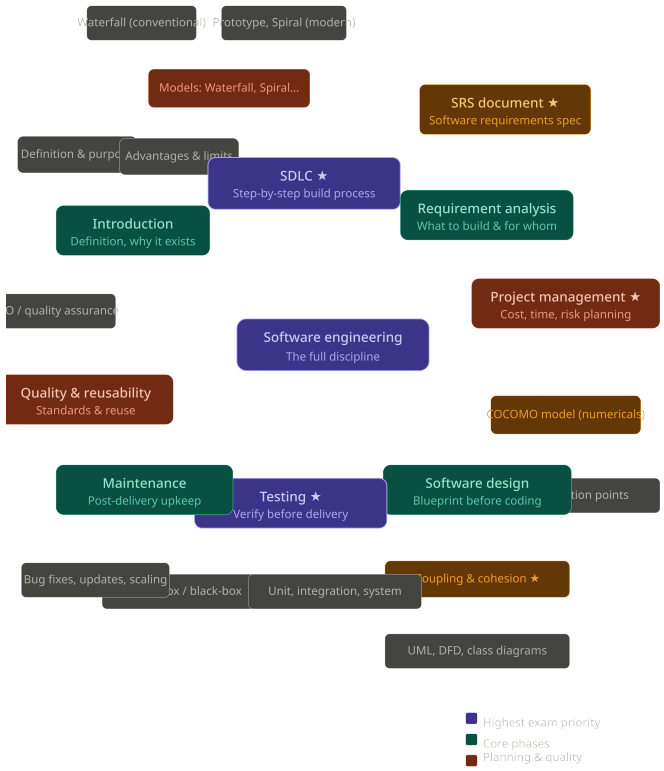
Introduction

1.1 Syllabus Overview

Software engineering is the disciplined, step-by-step process of taking a software idea from conception all the way to a maintained, quality product — and this notes covers every major phase of that journey.

ELI5:-

Imagine you and your friends want to build the coolest treehouse ever. You can't just grab wood and nails and hope for the best. First you talk about what you want (requirements). Then you draw a plan (design). Then you actually build it (coding). Then you check if it's safe and fun (testing). Then you fix stuff when a plank breaks later (maintenance). And you make sure it's sturdy enough to last (quality). Software engineering is learning all those steps properly — so your treehouse doesn't fall down on day one.



Engineering, in any discipline, means applying proven principles to build something reliably and systematically. Mechanical engineering does this for machines. Civil engineering does this for structures. Software engineering does this for software — taking an idea from someone's head and turning it into a working, maintainable product through a defined process.

This syllabus is 80–85% theoretical, with a small numerical portion. The theoretical weight does not mean it is unimportant — in fact, software engineering concepts directly govern how every real software company operates. The lectures approach all theory through real-life examples to stop it from feeling abstract.
Here is everything the syllabus covers, in order:

1. **Introduction** - What software engineering is, why it exists, its advantages and disadvantages, basic applications, and the difference between a 'software' and a 'project'. The core idea: just as mechanical engineering gives you the principles to design machines, software engineering gives you the principles to design software systems.

2. **Software Development Life Cycle (SDLC) — most important topic** - DLC is the step-by-step recipe for building software. It tells you what to do first, second, third — so that the final product matches what the client wanted, on time and within budget. The specific phases inside SDLC are covered individually below (requirement, design, implementation, testing, maintenance). SDLC questions are guaranteed in both competitive and university exams.

3. **DLC Models — conventional and modern** - These are specific ways of executing the SDLC phases. Conventional models include the Waterfall model (strict, linear, one phase must finish before the next begins). Modern/iterative models include the Incremental model, Prototype model, and Spiral model. Each model is a different philosophy about how to handle uncertainty and change during software development.

4. **Requirement Analysis** - Before building anything, you must gather exactly what needs to be built. This phase produces the SRS — Software Requirements Specification — a formal document that acts as a contract between the client and the development team. It captures functional and non-functional requirements with enough detail that no ambiguity remains. SRS preparation is asked in placements too.

5. **Software Project Management** - Once requirements are clear, you must plan the project: how long will it take, how much will it cost, what can go wrong. This phase covers cost estimation (using metrics like Lines of Code and Function Points), time estimation, and risk analysis. The COCOMO model is the key numerical topic here and is heavily asked in competitive and university exams. The analogy: before building a house, you estimate material costs, labour time, and risks like floods or earthquakes in that area not after.

6. **Software Design** - Before coding begins, a blueprint must be created. Software designers produce DFDs (Data Flow Diagrams), UML diagrams, and class diagrams that specify how the system will be structured. The two most exam-important design concepts are coupling (how much modules depend on each other — lower is better) and cohesion (how focused each module is on one job — higher is better). These are also directly used when writing project reports and research reports.

7. **Implementation (Coding)** - The actual construction phase — translating the design into working code. Software engineering does not teach you to code here; it tells you that this phase comes after design is locked, and governs how it fits into the larger process.

8. **Testing (2nd most important topic)** - After implementation, the software must be verified. Testing types covered include: unit testing (individual components), integration testing (components working together), system testing (the full system), white-box testing (testing internal logic — you can see the code), and black-box testing (testing external behaviour — you cannot see the code). Testing questions are guaranteed in exams, almost on par with SDLC.

9. **Maintenance** - Delivery is not the end. After a software product is handed to the client, problems will still surface — bugs, changing requirements, performance issues, security patches. The maintenance phase handles all of this. It is either done by the original development team (often specified in the SRS) or by the client's own team. Real example from the lecture: the Gate Smashers test portal itself requires ongoing maintenance calls to its developers whenever something breaks.

10. **Quality Management** - The software must meet defined quality standards — comparable to ISO certification for a product. Quality is not tested at the end; it is managed throughout the process. Concepts here include software quality attributes and quality assurance frameworks.

11. **Reusability** - Once a software component or system is built, it should ideally be reusable in future projects. Designing for reuse saves cost and time. The analogy: a house you no longer live in can be converted into a PG hostel — the investment is reused rather than discarded.

All of these topics will be covered with real-life examples throughout the notes, because the best way to internalize a theoretical subject is to anchor every concept to something you have already experienced."

Real World Analogy:-

The entire software engineering syllabus maps perfectly onto building a house. You get an idea for the house (introduction). You follow a defined construction sequence (SDLC). You choose a construction style — build everything at once or floor by floor (SDLC models). You item every requirement — land, money, workers, room sizes (requirement analysis). You estimate total cost, time, and check for flood earthquake risks before breaking ground (project management). You sit with an architect and produce a blueprint (software design). The workers then actually build (implementation). Before you move in, you inspect every room and check the plumbing (testing). After you move in, you repaint walls and fix leaky taps as they appear (maintenance). And you make sure the house is certified safe to live in (quality management).

Connections :-

1. SDLC Models (Waterfall, Spiral, Prototype, Incremental): These are the implementation strategies for the SDLC — each one decides how you sequence and repeat the phases depending on project type and risk tolerance.
2. SRS Document: The formal output of requirement analysis. Understanding SRS well also directly helps with writing project reports and research methodology documents.
3. COCOMO Model: The numerical heart of software project management. It estimates effort and cost from lines of code or function points — one of the few places in SE where you do actual calculations.
4. Coupling and Cohesion: Core design metrics that determine how maintainable and testable a system is. Low coupling and high cohesion are the design goals every software architect aims for.
5. Testing types (White-box, Black-box, Unit, Integration, System): Each type targets a different level of the system and a different kind of defect. Together they form the verification layer of SDLC.
6. UML and DFD: Diagramming tools used in the design phase. Also appear in research and project documentation — directly relevant to thesis and project report work.

Question 1

Walk through the entire SDLC using the house-building analogy — one sentence per phase

Solution: todo

Question 2

The lecture says software engineering is 80–85% theoretical. Does that mean it is less important for a working developer? Make the case for why a coder still needs to understand SDLC.

Solution: todo

Question 3

What is the difference between SDLC (the process) and an SDLC model (like Waterfall)? Why does this distinction matter when choosing how to run a project?

Solution: todo

Question 4

Why is risk analysis done during project management rather than after the software is built? What is the cost of discovering a risk late?

Solution: todo

Question 5

If a client says 'just build it, we'll figure out the requirements as we go' — which SDLC model best handles that, and why would Waterfall be a bad choice here?

Solution: todo

Question 6

Explain the difference between white-box and black-box testing to someone who has never heard of either.

Solution: todo

Common Misconceptions :-

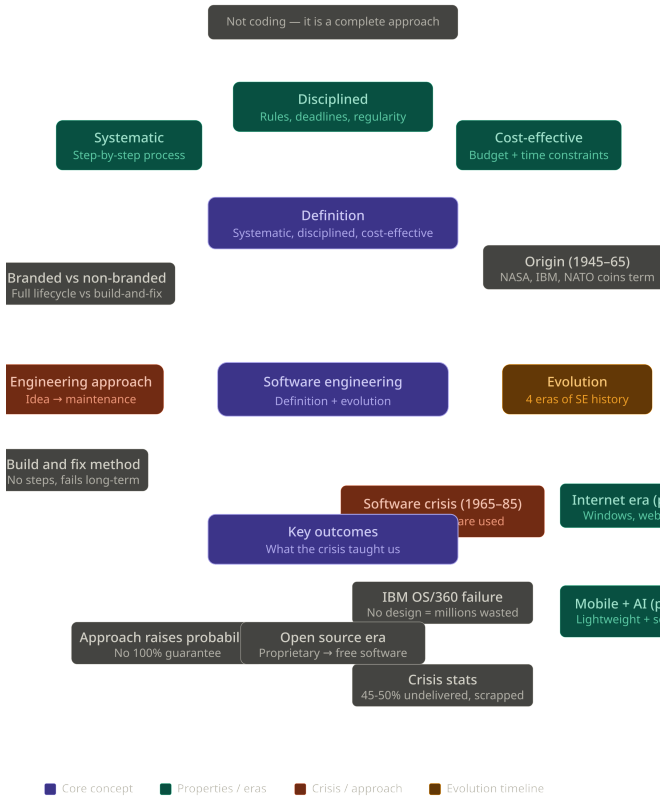
1. SDLC and Waterfall are not the same thing. SDLC is the general framework of phases every software project follows. Waterfall is one specific model that executes those phases in a strict linear order. Spiral, Prototype, and Incremental are other models — all still follow SDLC, just differently.
2. Maintenance is not a sign of failure. Many students assume that if a product needs maintenance, something went wrong. In reality, maintenance is a planned, expected, and often contractually specified phase. Every non-trivial software system requires ongoing maintenance — it is built into the project budget and SRS from day one.
3. Software engineering is not just for developers. The syllabus covers roles across testing, design, project management, and quality assurance. Even if you never write a line of production code, understanding SE is essential for any technical role in a software company.

1.2 What is software engineering and its evolution

Software engineering is a systematic, disciplined, cost-effective approach to building software — and it evolved over decades from chaotic 'build and deliver' hacking into a structured discipline after a period of catastrophic failures known as the software crisis.

ELI5:-

Imagine two kids making a sandwich. One just grabs whatever is in the fridge, slaps it together, and hands it to you — it might taste okay or it might be awful, and if you want another one tomorrow, good luck. The other kid follows a recipe, checks the ingredients, times everything, and even cleans up after. Software engineering is teaching everyone to be the second kid — not just making software, but making it properly, step by step, on time, and within budget.



Software engineering is defined as a systematic, disciplined, and cost-effective technique for software development. Breaking that down: systematic means following a step-by-step process rather than winging it; disciplined means working consistently under deadlines and following defined rules; cost-effective means staying within budget and time constraints. The key insight is that software engineering is NOT about coding — coding is just one phase. The discipline covers the entire journey from idea to maintenance.

The second, complementary definition is: 'an engineering approach to develop software.' Engineering approach does not mean 'work done by engineers' — it means achieving a goal step by step, systematically, from first step to last. The branded-vs-non-branded analogy illustrates this perfectly. A local mechanic modifies a Jeep and sells it with no testing, no guarantee, and no after-sales support (build and deliver). A company like Mahindra or Toyota follows every step rigorously, tests the product thoroughly, and still provides maintenance years after delivery. That full lifecycle commitment is the engineering approach.

The evolution of software engineering spans four eras. The origin phase (1945–1965) began with NASA, IBM, and defence organisations building software with no formal methodology — just 'build and fix.' The term 'software engineering' itself was first coined at NATO conferences during this period. Then came the software crisis (1965–1985): without structured approaches, projects were failing at alarming rates — only 2% of software built was actually used as intended, 45–50% was never delivered, and most of the rest was scrapped due to budget or time overruns. IBM's OS/360 is a famous example — the project head himself called it a 'multi-millionaire failure' because no coherent design was prepared before coding began. The Internet era (post-1990) brought scale and legitimacy — Windows, web browsers, and networked systems demanded real engineering rigour. The mobile era then required lightweight, platform-specific development. And the current AI/ML era (post-2010) introduced self-learning systems — software that improves from data rather than purely explicit instructions.

A key outcome of this evolution is open-source culture — most software today is freely available, enabling anyone to learn, contribute, and build on existing work. This shift from proprietary-only to open ecosystems is itself part of what makes modern software engineering more accessible and collaborative.

Real World Analogy:-

The entire history of software engineering mirrors the history of manufacturing. Early factories had no quality control — products were made and shipped with no testing, no standard parts, no warranty. Factory disasters

and product failures forced the industry to adopt standards, quality checks, and formal processes. Software went through the same painful lesson between 1965 and 1985: build-and-deliver produced a graveyard of failed projects and wasted millions. Just as ISO standards and Six Sigma emerged to fix manufacturing, structured software engineering methodologies emerged to fix software development. The 'software crisis' was the industry's equivalent of a factory fire — painful, costly, but ultimately the forcing function that made the discipline take itself seriously.

Connections :-

1. SDLC: The direct answer to the software crisis. SDLC is the formalised step-by-step process that replaced 'build and fix' — it is what systematic and disciplined look like in practice.
2. Software project management and COCOMO: Cost-effectiveness isn't accidental — it requires upfront estimation of cost, time, and risk. COCOMO is the numerical model built to do exactly that.
3. Software testing: The build-and-deliver era had no testing phase. The OS/360 failure happened because testing was skipped. Formal testing methods are a direct response to the software crisis.
4. Open source / free software movement: The shift from proprietary to open software is the most visible outcome of SE's maturation — it is mentioned explicitly as a hallmark of the current era.
5. AI and machine learning: The current frontier of software engineering — systems that learn from data rather than following purely hard-coded instructions, representing the newest phase of the evolution timeline.

Question 7

Explain the definition of software engineering — specifically what 'systematic', 'disciplined', and 'cost-effective' each mean — to someone who thinks software engineering just means writing code.

Solution: todo

Question 8

The lecture uses the branded-vs-non-branded vehicle analogy. Map each vehicle scenario (local mechanic vs. Mahindra showroom) to a specific software engineering concept.

Solution: todo

Question 9

Only 2% of software built during the crisis era was actually used. What structural problem caused this, and how did formalised software engineering address it?

Solution: todo

Question 10

Why is the IBM OS/360 project considered a landmark example of the software crisis? What specific mistake made it fail?

Solution: todo

Question 11

Trace the four eras of software engineering evolution in order. For each era, name one software category or technology that is characteristic of it.

Solution: todo

Question 12

The lecture says following the engineering approach does not guarantee success — only that the probability of success is higher. Why is this honest framing important for a software engineer to internalise?

Solution: todo

Common Misconceptions :-

1. Software engineering is not just coding. The most common misconception is treating SE as synonymous with programming. Coding is one phase — implementation — inside a much larger process. SE covers requirement gathering, feasibility, design, testing, maintenance, and quality management. A project can be beautifully coded and still fail if requirements were gathered wrong or testing was skipped.
2. 'Engineering approach' does not mean 'work done by engineers.' The lecture explicitly addresses this. Engineering approach means achieving a goal through a systematic, step-by-step process. A mechanic who follows every step rigorously is using an engineering approach; an engineer who skips design and goes straight to coding is not.
3. The software crisis did not end in 1985. The lecture notes that software failures continued well beyond the defined crisis period — the Taurus project (UK) and multiple US autonomous system projects also failed from budget overruns and lack of structured approach, years after the crisis era. The discipline improved, but project failures never fully stopped.

Note:-

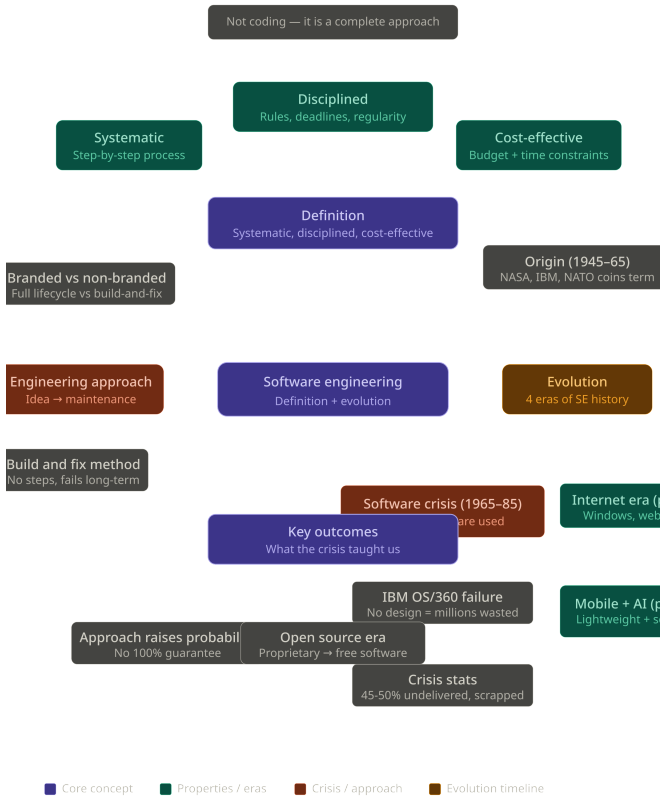
1. The build-and-fix method (1945–65) is the exact opposite of SDLC — it's the failure mode that SDLC was invented to solve. When you study SDLC models, keep this in the back of your head — every model exists because someone got burned skipping steps.
2. The OS/360 stat (multi-millionaire failure, no design prepared) is a high-yield exam anecdote. Examiners love asking "give an example of the software crisis" — that's your answer.

1.3 Software Development Life Cycle (SDLC) — phases with real-life examples

SDLC is the structured sequence of phases — planning, defining, designing, coding, testing, deployment, and maintenance — that every software project must pass through to be built properly, on time, and within budget.

ELI5:-

Imagine you want to open a lemonade stand. First you plan what you need (lemons, sugar, a table). Then you write it all down so nobody forgets. Then you decide how the stand should look. Then you actually build it. Then you taste the lemonade to make sure it's good. Then you open for business. And then you keep fixing things when the table wobbles or the lemonade runs out. SDLC is exactly those steps — but for making software.



SDLC exists because two parties are always involved in software development: the customer (who has an idea and money) and the service provider (who has the skills and team). Without a defined process, neither party has any guarantee that the final product will match what was asked for, cost what was agreed, or arrive on time. SDLC is the contract between those two parties — made visible as a sequence of phases.

The first phase is planning. The customer articulates what they want — features, platforms, rough budget, timeline. Multiple service providers submit quotations. This is pure ideation: dashboards, login flows, content types, landing pages all exist only in the customer's head at this point.

The second phase is defining or analysis. The service provider takes the customer's verbal requirements and analyses them internally — estimating cost using COCOMO and function points, estimating staff and time, identifying risks. Negotiation happens here. The output is the SRS document — Software Requirements Specification — a formal written agreement that captures everything both parties agreed to. The SRS is critical: it locks the scope so that neither side can be blindsided later. Customers can still request changes after this, but changes cost money and time.

The third phase is designing. The service provider builds the blueprint — DFDs, UML diagrams, ER models, module structures. Key design concepts like cohesion (how focused each module is) and coupling (how dependent modules are on each other) are decided here. No code is written yet. This is the architect's drawing before construction begins.

The fourth phase is implementation — coding. Developers pick the tech stack (language, backend framework, database), and build the system from the design documents. The design phase makes this phase straightforward: there are no open questions, just execution.

The fifth phase is testing. Before the product goes to users, it must be verified. Testing types include unit testing (individual components), integration testing (components together), system testing (the full system), and alpha/beta testing (pre-release user testing). White-box and black-box methods apply here. Bugs found in testing are far cheaper to fix than bugs found by real users after launch.

The sixth phase is deployment. The product is handed to the customer, who receives a full demo and training on how to operate it. This is not the end of the service provider's job.

The seventh phase is maintenance. A standard contract includes 3–4 months of free support after delivery — bug fixes, minor changes, handholding. After that, maintenance charges apply on a monthly or yearly basis. The

cycle then repeats for the next version or project.
SDLC is not just exam theory. Any startup, entrepreneurship journey, or software company role puts you inside one of these phases. Understanding SDLC tells you how to manage vendors, structure meetings, communicate requirements, and know when to push back on scope creep.

Real World Analogy:-

SDLC maps exactly to commissioning a custom-built house. You tell an architect what you want (planning). The architect draws up a blueprint after detailed discussions and you both sign off on it (defining + SRS). The architect finalises the floor plan, electrical layout, and structural design (designing). Contractors build the house (implementation). A building inspector walks through every room before you get the keys (testing). You move in (deployment). And you call a plumber when something leaks six months later — at your own cost after the warranty period (maintenance). A software project without SDLC is like hiring a contractor with no blueprint and hoping for the best.

Connections :-

1. SRS document: The tangible output of the defining/analysis phase. Every requirement that isn't in the SRS is invisible to the service provider — ambiguity here causes the most expensive bugs.
2. COCOMO model: The numerical engine behind cost and time estimation in the analysis phase. It converts scope into numbers the service provider can quote.
3. Coupling and cohesion: The core design-quality metrics decided in the designing phase. They determine how maintainable and testable the system will be long before a line of code is written.
4. Software testing types (unit, integration, system, white-box, black-box): Each type of test catches a different category of defect at a different scale of the system.
5. SDLC models (Waterfall, Prototype, Spiral, RAD, V-model, Agile): These are different ways of sequencing and iterating through the SDLC phases — the phases themselves remain the same, but the order and repetition change.

Question 13

Walk through all 7 SDLC phases using the Gate Smashers learning platform example from the lecture — one sentence per phase.

Solution: todo

Question 14

Why is the SRS document produced in the analysis phase rather than the planning phase? What specifically triggers its creation?

Solution: todo

Question 15

The lecture mentions that customers can still request changes after the SRS is signed. What problem does this cause, and how does SDLC help manage it?

Solution: todo

Question 16

What is the designing phase actually producing, and why does it come before implementation instead of being done alongside it?

Solution: todo

Question 17

A service provider gives 3–4 months of free maintenance after deployment. What is the business logic behind this, and what happens after that period ends?

Solution: todo

Question 18

SDLC models like Waterfall and Agile are described as 'different ways of implementing SDLC.' What does that mean — what exactly is different between them if all models use the same phases?

Solution: todo

Common Misconceptions :-

1. The customer's job does not end at the SRS. A common mistake is thinking the customer hands over requirements and disappears. In reality, customers stay involved — particularly in the designing phase (approving wireframes and layouts) and testing phase (alpha/beta feedback). The SRS sets the baseline, but collaboration continues.
2. Deployment is not the last phase. Students often treat delivery as the finish line. Maintenance is a formal, contractually defined phase with its own cost structure. In real projects, maintenance can last years and consumes significant budget — sometimes more than the original build.
3. SDLC is not a model — it is the framework. Waterfall, Agile, Spiral, RAD, and V-model are all SDLC models. They differ in how they sequence, overlap, or repeat the phases. SDLC itself is the collection of phases that every model draws from.

Chapter 2

2.1 Random Examples

ELI5:-

this is test

Real World Analogy:-

this is also test

Further Readings :-

this is also test

Question 1

this is test
this is test
this is test

Common Misconceptions :-

this is test

Connections :-

this is test

Definition 2.1.1: Limit of Sequence in $\mathbb{R}$

Let $\{s_n\}$ be a sequence in $\mathbb{R}$. We say

$$\lim_{n\to\infty} s_n = s$$

where $s \in \mathbb{R}$ if $\forall$ real numbers $\epsilon > 0$ $\exists$ natural number N such that for $n > N$

$$s - \epsilon < s_n < s + \epsilon \text{ i.e. } |s - s_n| < \epsilon$$

Question 2

Is the set $x-$axis$\backslash\{$Origin$\}$ a closed set

Solution: We have to take its complement and check whether that set is a open set i.e. if it is a union of open balls

Note:-

We will do topology in Normed Linear Space (Mainly $\mathbb{R}^n$ and occasionally $\mathbb{C}^n$)using the language of Metric Space

Claim 2.1.1 Topology

Topology is cool

Example 2.1.1 (Open Set and Close Set)

Open Set:
- ϕ
- $\bigcup_{x \in X} B_r(x)$ (Any $r > 0$ will do)
- $B_r(x)$ is open

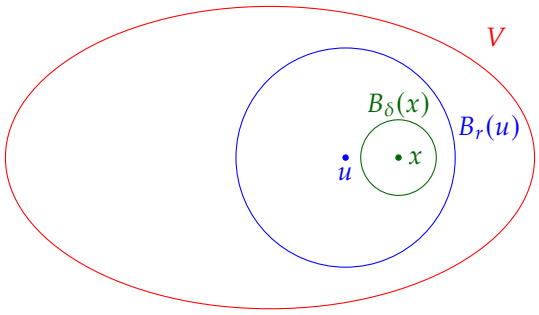
Closed Set:
- $X,\ \phi$
- $\overline{B_r(x)}$

$x-$axis $\cup$ $y-$axis

Theorem 2.1.1

If $x \in$ open set V then $\exists\ \delta > 0$ such that $B_\delta(x) \subset V$

Proof: By openness of V, $x \in B_r(u) \subset V$



Given $x \in B_r(u) \subset V$, we want $\delta > 0$ such that $x \in B_\delta(x) \subset B_r(u) \subset V$. Let $d = d(u, x)$. Choose δ such that $d + \delta < r$ (e.g. $\delta < \frac{r-d}{2}$)

If $y \in B_\delta(x)$ we will be done by showing that $d(u, y) < r$ but

$$d(u, y) \leqslant d(u, x) + d(x, y) < d + \delta < r$$

Corollary 2.1.1

By the result of the proof, we can then show...

Suppose $\vec{v}_1, \ldots, \vec{v}_n \in \mathbb{R}^n$ is subspace of $\mathbb{R}^n$.

Proposition 2.1.1

$1 + 1 = 2$.

2.2 Random

Definition 2.2.1: Normed Linear Space and Norm $\|\cdot\|$

Let V be a vector space over $\mathbb{R}$ (or $\mathbb{C}$). A norm on V is function $\|\cdot\| \; V \to \mathbb{R}_{\geqslant 0}$ satisfying

① $\|x\| = 0 \iff x = 0 \; \forall \; x \in V$

② $\|\lambda x\| = |\lambda|\|x\| \; \forall \; \lambda \in \mathbb{R}(\text{or } \mathbb{C}), \; x \in V$

③ $\|x + y\| \leqslant \|x\| + \|y\| \; \forall \; x, y \in V$ (Triangle Inequality/Subadditivity)

And V is called a normed linear space.
• Same definition works with V a vector space over $\mathbb{C}$ (again $\|\cdot\| \to \mathbb{R}_{\geqslant 0}$) where ② becomes $\|\lambda x\| = |\lambda|\|x\|$ $\forall \; \lambda \in \mathbb{C}$, $x \in V$, where for $\lambda = a + ib$, $|\lambda| = \sqrt{a^2 + b^2}$

Example 2.2.1 ($p-$Norm)

$V = \mathbb{R}^m$, $p \in \mathbb{R}_{\geqslant 0}$. Define for $x = (x_1, x_2, \cdots, x_m) \in \mathbb{R}^m$

$$\|x\|_p = \left(|x_1|^p + |x_2|^p + \cdots + |x_m|^p\right)^{\frac{1}{p}}$$

(In school $p = 2$)

Special Case $p = 1$: $\|x\|_1 = |x_1| + |x_2| + \cdots + |x_m|$ is clearly a norm by usual triangle inequality.
Special Case $p \to \infty$ ($\mathbb{R}^m$ with $\|\cdot\|_\infty$): $\|x\|_\infty = \max\{|x_1|, |x_2|, \cdots, |x_m|\}$
For $m = 1$ these $p-$norms are nothing but $|x|$. Now exercise

Question 3

Prove that triangle inequality is true if $p \geqslant 1$ for $p-$norms. (What goes wrong for $p < 1$?)

Solution: **For Property ③ for norm-2**

When field is $\mathbb{R}$:

We have to show

$$\sum_i (x_i + y_i)^2 \leqslant \left(\sqrt{\sum_i x_i^2} + \sqrt{\sum_i y_i^2}\right)^2$$

$$\implies \sum_i (x_i^2 + 2x_i y_i + y_i^2) \leqslant \sum_i x_i^2 + 2\sqrt{\left[\sum_i x_i^2\right]\left[\sum_i y_i^2\right]} + \sum_i y_i^2$$

$$\implies \left[\sum_i x_i y_i\right]^2 \leqslant \left[\sum_i x_i^2\right]\left[\sum_i y_i^2\right]$$

So in other words prove $\langle x, y\rangle^2 \leqslant \langle x, x\rangle\langle y, y\rangle$ where

$$\langle x, y\rangle = \sum_i x_i y_i$$

Note:-

- $\|x\|^2 = \langle x, x\rangle$
- $\langle x, y\rangle = \langle y, x\rangle$

- $\langle \cdot, \cdot \rangle$ is $\mathbb{R}-$linear in each slot i.e.

$$\langle rx + x', y \rangle = r\langle x, y \rangle + \langle x', y \rangle \text{ and similarly for second slot}$$

Here in $\langle x, y \rangle$ x is in first slot and y is in second slot.

Now the statement is just the Cauchy-Schwartz Inequality. For proof

$$\langle x, y \rangle^2 \leqslant \langle x, x \rangle \langle y, y \rangle$$

expand everything of $\langle x - \lambda y, x - \lambda y \rangle$ which is going to give a quadratic equation in variable λ

$$\begin{aligned} \langle x - \lambda y, x - \lambda y \rangle &= \langle x, x - \lambda y \rangle - \lambda \langle y, x - \lambda y \rangle \\ &= \langle x, x \rangle - \lambda \langle x, y \rangle - \lambda \langle y, x \rangle + \lambda^2 \langle y, y \rangle \\ &= \langle x, x \rangle - 2\lambda \langle x, y \rangle + \lambda^2 \langle y, y \rangle \end{aligned}$$

Now unless $x = \lambda y$ we have $\langle x - \lambda y, x - \lambda y \rangle > 0$ Hence the quadratic equation has no root therefore the discriminant is greater than zero.

When field is $\mathbb{C}$:

Modify the definition by

$$\langle x, y \rangle = \sum_i \overline{x_i} y_i$$

Then we still have $\langle x, x \rangle \geqslant 0$

2.3 Algorithms

Algorithm 1: what

Input: This is some input
Output: This is some output
`/* This is a comment */`

```
1  some code here;
2  x ← 0;
3  y ← 0;
4  if x > 5 then
5  |  x is greater than 5 ;                    // This is also a comment
6  else
7  |  x is less than or equal to 5;
8  end
9  foreach y in 0..5 do
10 |  y ← y + 1;
11 end
12 for y in 0..5 do
13 |  y ← y − 1;
14 end
15 while x > 5 do
16 |  x ← x − 1;
17 end
18 return Return something here;
```